

17

LSTM

Long Short-Term Memory — networks that remember

Built for sequences: text, time, and anything in order

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A neural network with memory, for sequences
Full name	Long Short-Term Memory
Built for	Text, speech, time series — data with an order
Key idea	Remember important things, forget the rest
You've used it	Your Twitter sentiment project (Bidirectional LSTM)

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	Why a Network with Memory?	Sequences and why order matters
2	RNN — The First Try	A network that loops
3	The Forgetting Problem	Why simple RNNs fail
4	LSTM to the Rescue	Memory done right
5	The Three Gates	Forget, input, output
6	Cell State	The memory conveyor belt
7	Building an LSTM in Code	A real PyTorch example
8	Bidirectional LSTM	Reading both directions
9	GRU — The Lighter Cousin	A quick preview
10	Where LSTMs Are Used	Real jobs
11	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. Why a Network with Memory?

CNNs are great for images, where all the pixels are present at once. But some data comes as a **sequence** (■■■■■■■■■■ — ■■■■) — one piece at a time, in order. Sentences are sequences of words. Stock prices are sequences over time. Speech is a sequence of sounds.

For sequences, **order matters**. Look at these two sentences: 'the food was not good' and 'the food was good'. Just one word changes everything. A normal network that ignores order would miss this completely.

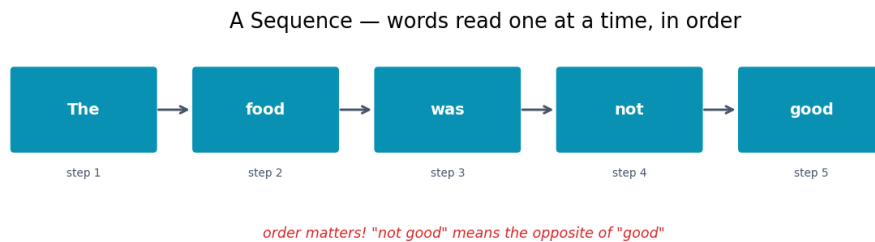


Fig 1 — A sentence is a sequence. The word 'not' flips the meaning — so order and memory matter.

To understand a sequence, a network needs **memory**: it must remember earlier parts while reading later parts. By the time it reaches 'good', it should remember it saw 'not' before. This is exactly what LSTMs are built to do.

■ *This is the network family behind your Twitter Airline Sentiment project. Understanding LSTM explains why your Bidirectional LSTM worked the way it did.*

2. RNN — The First Try

The first idea for handling sequences was the **RNN** (Recurrent Neural Network — ■■■■■■■■■■ ■■■■■■ ■■■■■■■■■■). 'Recurrent' means it loops. It reads the sequence one item at a time and passes a **memory** forward to the next step.

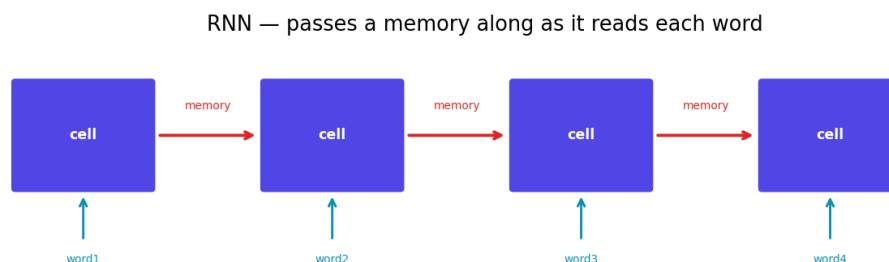


Fig 2 — An RNN reads each word and passes a memory along, so later steps know about earlier ones.

At each step, the RNN combines the new word with the memory from before, makes an updated memory, and passes it on. So information flows along the sequence. In theory, by the end it has 'read' the whole sentence and remembers what happened. In practice, there's a big problem.

3. The Forgetting Problem

Simple RNNs have a serious weakness: they **forget quickly**. By the time they read a long sentence, they've forgotten how it started. The memory of early words fades away after just a few steps.

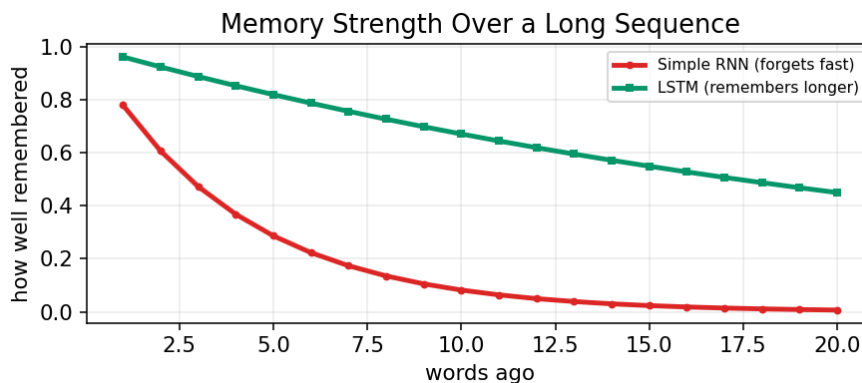


Fig 3 — A simple RNN's memory fades fast (red). The LSTM holds onto information much longer (green).

This is called the **vanishing gradient problem** (██████████ ███████████████████) — the learning signal shrinks as it travels back through many steps, until it's too tiny to teach the early parts anything. Imagine reading a long book but forgetting each page as soon as you turn it. You'd never understand the ending.

For a sentence like 'I grew up in France... so I speak fluent ____', a simple RNN would have forgotten 'France' by the time it needs to guess 'French'. We need a network that can hold onto important information for a long time. That's the LSTM.

4. LSTM to the Rescue

The **LSTM** (Long Short-Term Memory — ) is a smarter kind of RNN, designed specifically to remember things for a long time. The name says it all: it keeps both short-term and long-term memory, and decides what's worth keeping.

The key insight: instead of one big messy memory that gets overwritten every step, the LSTM has a separate, protected memory line — and three smart **gates** (■■■ — ■■■■) that carefully control what goes in, what stays, and what comes out. The gates are the secret.

■ Think of the LSTM like a careful note-taker. As it reads, it decides: 'this is important, keep it', 'this is junk, drop it', 'I need to use this now'. That selective memory is what makes it powerful.

5. The Three Gates

The LSTM's power comes from three gates. Each gate is a tiny decision-maker that controls the flow of memory. They work together at every step.

LSTM Gates — three smart controls for memory



Fig 4 — The three gates of an LSTM cell, each controlling a part of the memory.

Gate	What it decides	Everyday example
Forget gate	What to throw away from memory	Forget the old subject when a new one starts
Input gate	What new info to store	Remember a name just mentioned
Output gate	What to send out right now	Use the stored info to make a prediction

Each gate looks at the new word and the current memory, then outputs a value between 0 and 1 (using a sigmoid). A 0 means 'block completely', a 1 means 'let it all through'. By learning good gate values, the LSTM learns what to remember and what to forget — automatically, from data.

6. Cell State — The Memory Conveyor Belt

The LSTM keeps its long-term memory in something called the **cell state** (■■■■ ■■■■■■). Picture it as a conveyor belt running straight through the whole sequence. Information can ride along it almost unchanged for a long time.

The gates make small, careful edits to this belt: the forget gate removes some old items, the input gate adds some new ones. Because the belt runs straight through with only gentle changes, important information can travel a long way without fading. This is the trick that solves the forgetting problem.

■ So an LSTM cell carries two things forward: the cell state (long-term memory, the belt) and the hidden state (short-term working memory, the output at each step). The gates manage both.

7. Building an LSTM in Code

PyTorch makes LSTMs easy — one layer handles all the gate math for you:

```
import torch.nn as nn

class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim=100, hidden=128):
        super().__init__()
        # Turn word IDs into vectors the network understands
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        # The LSTM layer (handles all gates internally)
        self.lstm = nn.LSTM(embed_dim, hidden, batch_first=True)
        # Final decision layer: positive or negative
        self.fc = nn.Linear(hidden, 2)

    def forward(self, x):
        x = self.embedding(x)          # words -> vectors
        output, (hidden, cell) = self.lstm(x)
        last = hidden[-1]               # final memory
        return self.fc(last)           # classify

model = SentimentLSTM(vocab_size=10000)
```

■ *Notice `nn.Embedding`: text must become numbers first. An embedding turns each word into a vector of numbers that captures its meaning. This is the standard first layer for text models.*

8. Bidirectional LSTM — Reading Both Ways

Normally an LSTM reads left to right. But sometimes the meaning of a word depends on what comes AFTER it too. A **Bidirectional LSTM** (■■■■■■■■■■■■■■■■■■■■) reads the sequence both ways — forward and backward — and combines them. This gives a fuller understanding of each word's context.

```
# Just add bidirectional=True
self.lstm = nn.LSTM(embed_dim, hidden,
                    batch_first=True,
                    bidirectional=True)  # read both directions

# The output is now twice as wide (forward + backward)
self.fc = nn.Linear(hidden * 2, 2)
```

■ *This is exactly what you used in your Twitter sentiment project. Reading tweets both forward and backward helps catch context like sarcasm and negation ('not good') more reliably.*

9. GRU — The Lighter Cousin

The **GRU** (Gated Recurrent Unit — ) is a simpler, faster version of the LSTM. It does almost the same job but with only two gates instead of three, making it lighter and quicker to train. We'll cover it fully in the next topic.

	LSTM	GRU
Gates	Three (forget, input, output)	Two (reset, update)
Memory	Two states (cell + hidden)	One combined state
Speed	Slower (more to compute)	Faster
Performance	Slightly better on long sequences	Often just as good, simpler

In practice, people often try both and keep whichever works better. GRU is a good default when you want speed; LSTM is a good default when sequences are very long. You used both families in your sentiment work, so you've already seen this trade-off in action.

10. Where LSTMs Are Used

Job	How LSTM helps
Sentiment analysis	Read a review/tweet, decide positive or negative
Text generation	Predict the next word, one at a time
Machine translation	Read a sentence, output it in another language
Speech recognition	Turn a sequence of sounds into words
Time series forecasting	Predict next values from past values (sales, weather)

An honest note: for text and language, LSTMs have largely been replaced by **Transformers** (coming up soon), which are even better at long sequences and faster to train. But LSTMs are still widely used, easier to understand, and great for smaller projects and time series. Learning them builds the foundation for understanding Transformers.

11. Common Mistakes

Mistake	Why it's a problem	Fix
Forgetting the embedding	Text isn't numbers yet	Add nn.Embedding first
Sequences different lengths	Crash or bad batches	Pad sequences to equal length
Very long sequences	Slow, may still forget	Use Transformers, or trim length
Wrong output shape	Mismatch with classifier	Use the final hidden state
Not using bidirectional	Misses future context	Add bidirectional=True for text
Too high learning rate	Unstable training	Lower lr (try 0.001)

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
Sequence	data in order (words, time steps)
RNN	loops, passes memory forward
Forgetting problem	simple RNNs lose early info
LSTM	RNN with gates that remember long-term
Forget gate	decides what to drop from memory
Input gate	decides what new info to store
Output gate	decides what to send out now
Cell state	the long-term memory belt
Embedding layer	nn.Embedding(vocab_size, embed_dim)
LSTM layer	nn.LSTM(in, hidden, batch_first=True)
Bidirectional	nn.LSTM(..., bidirectional=True)
GRU (lighter)	nn.GRU(in, hidden, batch_first=True)
Pad sequences	make all sequences the same length

The one idea to remember: an LSTM is a network that reads a sequence one step at a time and uses three gates to decide what to remember, forget, and output — so it can hold onto important information for a long time.

Next Topic: GRU (Gated Recurrent Unit) — the LSTM's faster, simpler cousin. We'll see exactly how it slims down the gates while keeping most of the power.